

Contrôle de connaissance

Unité 4CS02 – Cryptologie
Durée : 1h30

Les supports de cours ne sont pas autorisés.

Une page recto-verso manuscrite ou imprimée est autorisée.

Les dispositifs électroniques sont interdits.

Les réponses peuvent être rédigées en anglais.

La rédaction et la justification des réponses seront prises en compte dans la note.

Le barème est donné à titre indicatif.

⚠ Les encadrés “Qui a raison, qui a tort” font partis du barème et doivent faire l’objet d’une réponse argumentée. Sans explication valable, la note à la question sera 0.

1 Système de chiffrement par bloc

IDEA (International Data Encryption Algorithm) est un chiffrement symétrique par blocs, originellement présenté en 1990, et qui a notamment été utilisé dans PGP et OpenPGP. Il emploie des clés de 128 bits pour chiffrer des blocs de 64 bits. Le schéma IDEA est en partie basé sur une variante du mécanisme de Feistel, dont la fonction de tour est décrite dans la Figure 1.

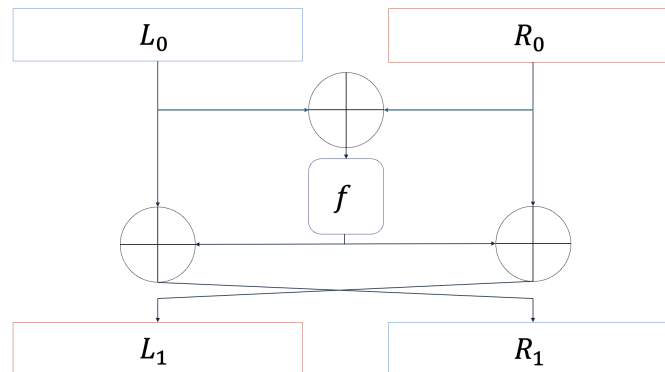


Figure 1: Structure du chiffrement IDEA

Question n°1 (3 points)

- On se concentre pour l’instant sur la fonction de chiffrement n’ayant qu’un tour. Écrire les équations donnant l’expression du chiffré (L_1, R_1) en fonction du clair (L_0, R_0) et de la fonction f .

- Montrer que ce schéma (on ne considère toujours qu'un tour) est inversible quelle que soit la fonction f et donner les formules décrivant le déchiffrement.
- Décrire un schéma de Feistel à trois tours qui est équivalent à ce tour d'IDEA. Vous aurez notamment à spécifier les trois fonctions f_1 , f_2 et f_3 de chacun des tours de Feistel.
- Montrer comment un attaquant est capable de distinguer un tour d'IDEA d'une fonction aléatoire. Pour cela, nous considérerons qu'il a accès à un oracle auprès duquel il peut envoyer un message (L_0, R_0) de son choix. L'oracle répondra systématiquement soit le chiffré résultant après un tour d'IDEA pour une fonction f secrète, soit une valeur aléatoire. Même question avec plusieurs tours d'IDEA. Est-ce que cela donne la capacité d'attaquer plusieurs tours de Feistel ?

Réponses

- $L_1 = R_0 \oplus f(L_0 \oplus R_0)$ et $R_1 = L_0 \oplus f(L_0 \oplus R_0)$.
- On remarque que $L_0 \oplus R_0 = L_1 \oplus R_1$ et donc $L_0 = R_1 \oplus f(L_1 \oplus R_1)$ et $R_0 = L_1 \oplus f(L_1 \oplus R_1)$.
- Trois tours de Feistel donnent $(L_1 = R_0, R_1 = L_0 \oplus f_1(R_0))$, puis $(L_2 = L_0 \oplus f_1(R_0), R_2 = R_0 \oplus f_2(L_0 \oplus f_1(R_0)))$ et enfin $(L_3 = R_0 \oplus f_2(L_0 \oplus f_1(R_0)), R_3 = L_0 \oplus f_1(R_0) \oplus f_3(R_0 \oplus f_2(L_0 \oplus f_1(R_0))))$. Cela donne directement que $f_1 = f_3 = Id$ et $f_2 = f$.
- Il suffit à nouveau d'utiliser le fait que $L_0 \oplus R_0 = L_1 \oplus R_1$. Si un attaquant envoie (L_0, R_0) à l'oracle, et obtient (L_1, R_1) en retour, il n'a qu'à vérifier si cette relation entre l'entrée et la sortie est vérifiée. Si c'est le cas, alors il s'agit probablement d'IDEA, et sinon, il s'agit forcément de la fonction aléatoire. Cela se généralise aisément à plusieurs tours. Par contre, cela ne donne en rien une attaque plusieurs tours de Feistel, puisqu'IDEA ne fournit qu'un cas particulier de Feistel avec $f_1 = f_3 = Id$ et $f_2 = f$.

Pour information (et ne pas vous laissez dans l'idée qu'un schéma autant utilisé est aussi mauvais), une fonction H sur l'ensemble du bloc est rajoutée entre chaque tour, afin de casser cette attaque.

Qui a raison, qui a tort n°1 (1 point) Alice pense qu'en implémentant la fonction H comme un xor avec une clé de tour et la fonction f telle que $f(B) = B \oplus b$ pour une valeur b publique, Shannon sera content d'elle. Bob pense qu'Alice a oublié une étape... Qui a raison, qui a tort ? Donnez votre réponse en argumentant.

Réponse. Bob a raison, il manque l'étape de confusion non-linéaire à la proposition d'Alice.

2 Chiffrement authentifié

La Figure 2 présente le mode GCM (pour un message composé de deux blocs de taille 128 bits). Dans cette figure, la valeur iv correspond à un vecteur d'initialisation de 128 bits (taille d'un bloc AES). La fonction $incr$ consiste à incrémenter d'une unité la donnée fournie en entrée. La fonction E_K correspond à l'algorithme AES avec une clé K partagée par les deux protagonistes et prend en entrée un message pour donner en sortie le chiffré correspondant. Le message donné en entrée est noté Plaintext. Il est découpé en plusieurs blocs (ici 2, avec donc Plaintext 1 et Plaintext 2) de taille 128 bits. La fonction $mult_H$ est une fonction particulière qui sera étudiée ci-dessous. Les valeurs AuthData, $len(A)$ et $len(C)$ sont des valeurs que l'on peut considérer comme publiques.

Si Alice exécute cet algorithme avec en entrée iv , AuthData, $len(A)$, $len(C)$ et le message (Plaintext 1 et Plaintext 2 de 128 bits chacun), elle obtiendra en sortie le chiffré composé de deux parties notées respectivement Ciphertext 1 et Ciphertext 2, chacun de 128 bits, et le tag AuthTag (aussi de 128 bits), qui seront tous envoyés à Bob, avec les valeurs iv , AuthData, $len(A)$ et $len(C)$.

Question n°2 (2 points)

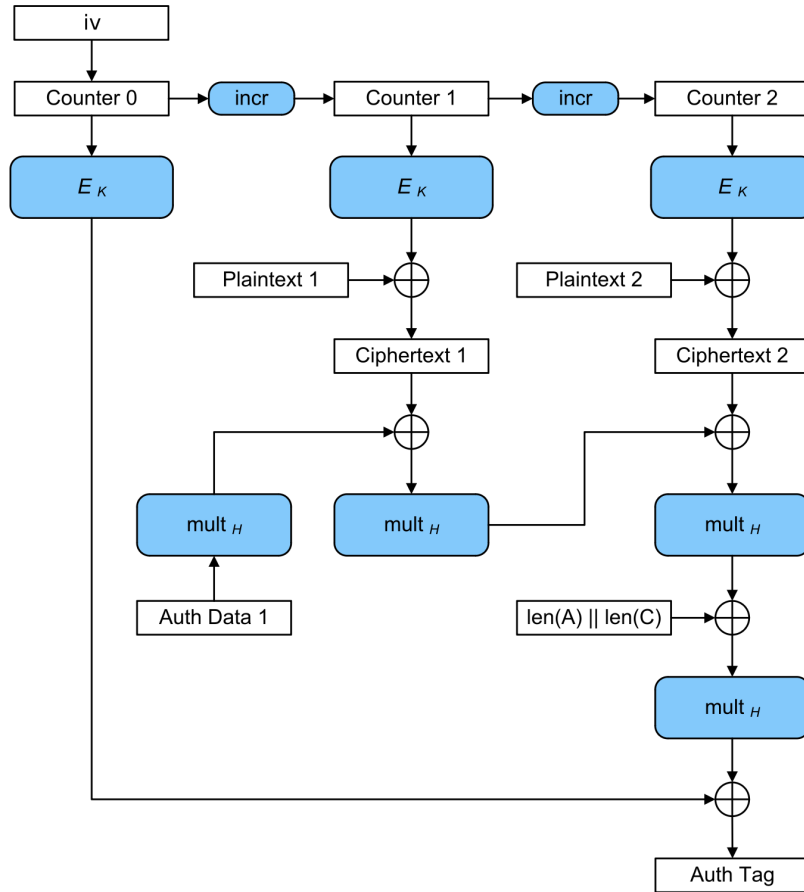


Figure 2: Structure du mode GCM

- Quelles sont les similitudes et les différences entre ce mode opératoire et la combinaison des modes CTR et CBC en mode *Encrypt-then-MAC* (mode habituellement appelé CCM) ? En particulier, quels sont les avantages de ce mode GCM ?
- Comment est-ce que Bob va devoir procéder pour retrouver le message (Plaintext 1 et Plaintext 2) et vérifier l'intégrité ?

Réponses

- GCM propose aussi un mécanisme interne à la *Encrypt-then-MAC*, avec un mode CTR pour la partie chiffrement, et quelque chose qui est très proche du mode CBC pour le calcul du MAC. Hormis ce dernier point, une différence de taille de GCM est que les deux modes se font en parallèle, alors qu'avec le mode CCM, il est obligatoire d'avoir fini la partie CTR avant de pouvoir faire le mode CBC. C'est l'un des gros avantages de GCM.
- Pour la vérification du MAC, il a juste à faire les calculs exactement dans le même sens qu'Alice, et vérifier ce qu'il obtient correspond à ce qu'Alice lui a envoyée. Ensuite, pour le déchiffrement, Bob connaissant iv , il peut recalculer les fonctions $K_i = E_K(\text{counter } i)$ et ensuite calculer simplement $\text{Plaintext } i = K_i \oplus \text{Ciphertext } i$.

La fonction mult_H correspond au mode dit de "Galois" (le 'G' de GCM), qui consiste à travailler dans le corps du même nom : $\text{GF}(2^{128}) = \mathbb{Z}_2[X]/P$ où P est le polynome $P(X) = X^{128} + X^7 + X^2 + X + 1$.

En simplifiant un peu, l'entrée E de mult_H est représentée par 128 bits et multipliée par $H = E_K(0^{128})$, la multiplication s'effectuant dans $\text{GF}(2^{128})$, et où 0^{128} correspond à une suite de 128 bits égaux à 0.

Question n°3 (1 points)

- A un moment de son calcul, Alice doit utiliser $\text{AuthData} = X^{127} + X^5 + X + 1$ et $H = X^2 + X + 1$. Combien obtient-elle en sortie de mult_H ? Vous détaillerez vos calculs en mettant en évidence les astuces utilisées, du fait que vous travaillez dans $\text{GF}(2^{128})$.

Réponses

- En utilisant le fait que nous travaillons dans $\mathbb{Z}_2[X]$, les coefficients des polynômes sont donc dans $\mathbb{Z}_2$, et donc, pour tout i , $X^i + X^i = 0$. Ainsi, la multiplication donne dans un premier temps $X^{129} + X^{128} + X^{127} + X^7 + X^6 + X^5 + X^3 + 1$. Ensuite, comme nous travaillons modulo $P(X) = X^{128} + X^7 + X^2 + X + 1$, il faut soit faire la division euclidienne du polynôme par P , soit utiliser le fait que $P(X) = 0 \pmod{P(X)}$, et donc que $X^{128} = X^7 + X^2 + X + 1$ et $X^{129} = X^8 + X^3 + X^2 + X$. En simplifiant le tout, on obtient le résultat : $X^{127} + X^8 + X^6 + X^5$.

Qui a raison, qui a tort n°2 (1 point) Alice est convaincue qu'une fonction de hachage peut être utilisée pour réaliser un contrôle d'intégrité et une authentification, mais Bob pense que ce n'est pas vrai... Qui a raison, qui a tort ? Donnez votre réponse en argumentant.

Réponse. Une fonction de hachage seule ne peut permettre que le contrôle d'intégrité, mais en aucun cas une authentification. Par contre, il est possible d'utiliser un HMAC, qui s'appuie sur l'utilisation d'une fonction de hachage, pour permettre les deux. C'est donc Alice qui a raison.

3 Échange de message sécurisé

Nous nous plaçons dans un groupe $\mathbb{G}$. Soit g un générateur public de $\mathbb{G}$. Alice et Bob communiquent à l'aide du protocole suivant, dans lequel Alice souhaite envoyer le message m à Bob de façon confidentielle :

1. Alice choisit secrètement a et calcule $x = g^a$. Elle envoie x à Bob.
2. Bob choisit secrètement b et calcule $y = g^b$. Il envoie y à Alice.
3. Alice calcule $c = m \cdot y^a$. Elle envoie c à Bob. .

Question n°4 (1 points)

- Quels sont les deux schémas cryptographiques que vous reconnaissez dans ce protocole ?
- Expliquez comment Bob peut retrouver m à partir des informations dont il dispose.

Réponses

- On reconnaît l'échange de clé Diffie-Hellman dans la première partie, et la façon de chiffrer fait un peu penser à ElGamal.
- Bob n'aura qu'à calculer $m = c/x^b$.

Qui a raison, qui a tort n°3 (1 point) Alice et Bob doivent calculer une exponentiation dans le groupe $\mathbb{G} = \mathbb{Z}_n^*$ où n est un module public. Alice dit qu'elle va utiliser l'exponentiation modulaire rapide. Bob lui répond que comme il connaît la factorisation (p, q) de $n = p \cdot q$, il peut utiliser le théorème des restes chinois et qu'il sera alors forcément plus rapide... Qui a raison, qui a tort ? Donnez votre réponse en argumentant.

Réponse. Bob connaissant la factorisation (p, q) de n (par exemple si c'est lui qui déchiffre), il peut en effet utiliser le théorème des restes chinois pour calculer $c^d \pmod{p}$ et $c^d \pmod{q}$, puis associer les deux résultats. Comme p et q sont beaucoup plus petit que n , il gagnera en temps de calcul, et sera plus rapide. C'est donc lui qui gagne. Par contre, pour calculer $c^d \pmod{p}$ et $c^d \pmod{q}$, il peut (il faut) qu'il utilise l'algorithme d'exponentiation rapide...

Question n°5 (3 points)

- Eve a la capacité d'intercepter les messages qui circulent entre Alice et Bob et d'émettre des messages en se faisant passer pour Alice auprès de Bob et Bob auprès d'Alice. Expliquer comment Eve peut intervenir dans les échanges du protocole ci-dessus pour être capable elle aussi de calculer m .
- Améliorer le protocole d'Eve pour assurer qu'à la fin du protocole, Bob reçoit bien lui aussi le message d'Alice et ne se doute de rien.
- Que doivent faire Alice et Bob pour se protéger contre la menace Eve ?
- Que deviennent les trois étapes du protocole ci-dessus si Alice et Bob utilisent une courbe elliptique pour leur groupe $\mathbb{G}$? Quel serait l'avantage de faire ça ?

Réponses

- Eve peut ici faire une attaque *man-in-the-middle* assez classique. Elle se met en coupure entre Alice et Bob, choisit son propre aléa e , récupère x de Bob, y d'Alice et envoie sa propre valeur $z = g^e$ aux deux. Ainsi, Alice utilisera g^{ae} , qu'Eve pourra calculer grâce à x^e , et ainsi obtenir le message m .
- Eve partage aussi une clé avec Bob, g^{be} . Elle peut donc calculer $m \cdot g^{be}$ avec le message qu'elle a reçu d'Alice à l'étape précédente. Bob le déchiffre sans difficulté, pensant qu'il vient d'Alice. Eve a réussi son attaque sans être détectée ni par Alice, ni par Bob.
- Alice et Bob doivent utiliser une signature électronique pour authentifier et contrôler l'intégrité de leurs envois, et ainsi être sûr que ce qu'ils reçoivent provient bien de l'autre. Ce n'est cependant pas suffisant. Ils faut aussi que leurs signatures s'appuient sur une clé publique qui a été certifiée par une autorité de certification (une PKI).
- Lorsque l'on travaille sur des courbes elliptiques, le groupe utilisé devient additif. Alice calcule donc $X = a \cdot G$, Bob calcule $Y = b \cdot G$ et Alice pourra calculer le chiffré par $C = m + a \cdot Y$. L'utilisation des courbes elliptiques permet d'utiliser des clés plus courtes (et donc d'être plus performant) pour une sécurité équivalente.

Qui a raison, qui a tort n°4 (1 point) Bob prétend qu'une clé RSA de 2048 bits est plus sécurisée qu'une clé AES-256, car $2048 > 256$, et donc que RSA est toujours meilleur pour la sécurité à long terme, mais Alice n'est pas d'accord... Qui a raison, qui a tort ? Donnez votre réponse en argumentant.

Réponse. Cette discussion n'a pas tellement de sens. AES est, et ne doit être, sensible qu'à l'attaque par force brute sur la clé, qui nécessite aujourd'hui par exemple 256 bits de clé. Il est aussi tout à fait possible de faire une attaque par recherche exhaustive sur RSA mais celle-ci sera de toute façon efficace que d'utiliser

un algorithme de factorisation sur la clé. D'où la nécessité d'augmenter la taille de la clé par rapport à AES. Ceci étant dit, un RSA de 2048 bits n'est aujourd'hui pas suffisamment sécurisé, contrairement à AES avec 256 bits de clé. Donc, c'est Alice qui a raison.

Finalement, Alice et Bob, une fois leur protocole corrigé, se rendent compte qu'ils sont limités dans la taille des messages qu'ils peuvent chiffrer. Pour cela, ils décident de changer la façon dont c est calculé de la façon suivante :

$$c = m \oplus \mathcal{H}(y^a),$$

où $\mathcal{H}$ est une fonction de hachage.

Question n°6 (2 points)

- *Comment est-ce que Bob va maintenant pouvoir retrouver m ?*
- *En supposant qu'ils utilisent une fonction de hachage qui donne en sortie des éléments de 256 bits, est-ce qu'Alice et Bob ont augmenté la taille de leur message par rapport à la version précédente ?*
- *Est-ce que par ce moyen, Alice et Bob parviennent au secret parfait défini par Shannon ?*

Réponses

- Bob va simplement devoir calculer $c \oplus \mathcal{H}(x^b)$ pour retrouver m (grâce aux propriétés de l'opération $\oplus$).
- Pas vraiment, car ils sont limités par la taille de la sortie de la fonction de hachage, qui va être 256 ou éventuellement 512. Ils pouvaient monter plus haut avec la version initiale du protocole (jusqu'à 2048 par exemple).
- La clé est aussi longue que le message, Alice et Bob peuvent tout à fait réutiliser ce même protocole pour chacun de leur message (la clé va donc tout le temps être nouvelle, mais cela ne suffit pas. En effet, ils n'y parviennent pas parce que pour obtenir le secret parfait de Shannon, ce dernier préconise que la clé soit générée de façon parfaitement aléatoire, ce qui n'est pas le cas ici. Il s'agit de la sortie d'une fonction de hachage sur des données non aléatoires.

Qui a raison, qui a tort n°5 (1 point) *Alice recommande d'utiliser MD5 comme fonction de hachage. Bob dit qu'elle se trompe et prétend que la fonction SHA1 est mieux... Qui a raison, qui a tort ? Donnez votre réponse en argumentant.*

Réponse. Cette fois-ci, les deux ont torts. Il ne faut plus utiliser ni MD5, ni SHA1, qui ont toutes les deux été attaquées...

Alice et Bob décident alors de diviser leur message en blocs de taille fixe, et de remplacer la fonction de hachage par un HMAC qui va prendre en entrée y^a et un compteur qui sera incrémenté pour chaque bloc du message.

Question n°7 (2 points)

- *Quelle va être la taille de chaque bloc ? Est-ce que cela vous paraît cohérent avec ce qui est actuellement préconisé ? A quoi vous fait penser cette façon de procéder ?*
- *Pensez-vous qu'ils soient parvenus à leurs fins ? Pensez-vous que cette façon de procéder soit une bonne chose pour un produit commercial ? Argumentez.*
- *Que manque-t-il comme propriété(s) de sécurité à leur protocole actuel pour éviter toute attaque ? Que faudrait-il rajouter ?*

Réponses

- Chaque bloc va être de la taille de la sortie de la fonction de hachage, et donc 256 ou 512 selon le choix qui a été fait. Nous obtenons $c = m \oplus \text{HMAC}(y^a \parallel \text{counter})$. Oui, c'est cohérent niveau des tailles. Cela fait penser au mode CTR.
- Ils sont effectivement parvenus à leurs fins. C'est un mélange assez original de plein de choses qui sont préconisées (mode CTR, HMAC, Diffie-Hellman, ElGamal, ...). Il faut faire attention et demander l'avis d'un expert, ou mieux de l'ANSSI, avant de déployer un tel système.
- Leur système ne permet que la confidentialité des données. Il leur manque le contrôle d'intégrité et l'authenticité de leurs échanges.

Qui a raison, qui a tort n°6 (1 point) Bob propose une autre variante à Alice, qui consiste à exécuter deux fois le protocole ci-dessus pour obtenir $K = y^a$ et $\tilde{K} = g^{\tilde{a}}$ et calculer à chaque bloc $m \oplus \text{HMAC}(K, i) \oplus \text{HMAC}(\tilde{K}, i)$, mais cette dernière n'est pas convaincue... Qui a raison, qui a tort ? Donnez votre réponse en argumentant.

Réponse. Cette façon de faire ressemble étrangement à un Double X, comme ce qui a été fait avec le 2DES. Ce n'est donc pas une bonne chose. Alice a raison.